Relazione sul Funzionamento Interno di Python

Appunti di Programmazione

20 marzo 2026

1 Introduzione a Python

Python è un linguaggio di programmazione ad alto livello, interpretato e multi-paradigma. A differenza dei linguaggi puramente compilati (come il C), Python utilizza un sistema ibrido che garantisce portabilità e flessibilità.

2 Il Flusso di Esecuzione

Il passaggio dal codice sorgente all'esecuzione sulla CPU non è diretto, ma attraversa diverse fasi cruciali:

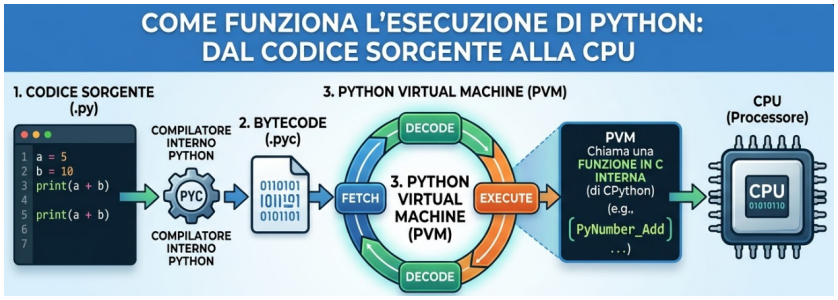


Figura 1: Funzionamento python con PVM

- **Compilazione in Bytecode** Quando un file `.py` viene eseguito, l'interprete (solitamente CPython) lo traduce in un formato intermedio chiamato **Bytecode**. Questo codice non è leggibile dall'uomo né direttamente dalla CPU, ma è ottimizzato per l'interprete. I file risultanti hanno spesso estensione `.pyc`.

- **La python Virtual Machine (PVM)** La PVM è il cuore del runtime di Python. Si tratta di un software che emula un microprocessore e agisce come strato di astrazione tra il Bytecode e l'hardware sottostante.

- **Il Meccanismo Interno: L'Interpreter Loop** La PVM opera attraverso un ciclo continuo di *Fetch-Decode-Execute*. Il dettaglio del funzionamento è il seguente:

  1. **Fetch:** La PVM preleva la successiva istruzione di Bytecode.
  2. **Decode:** L'istruzione viene identificata (es. `BINARY_ADD`).
  3. **Execute:** La PVM richiama una funzione interna scritta in **linguaggio C** (le cosiddette *C-API* di CPython).
  4. **Linguaggio Macchina:** La funzione C, essendo già compilata, viene eseguita direttamente dal processore come sequenza di istruzioni binarie.

2.1 Confronto tra C e Python

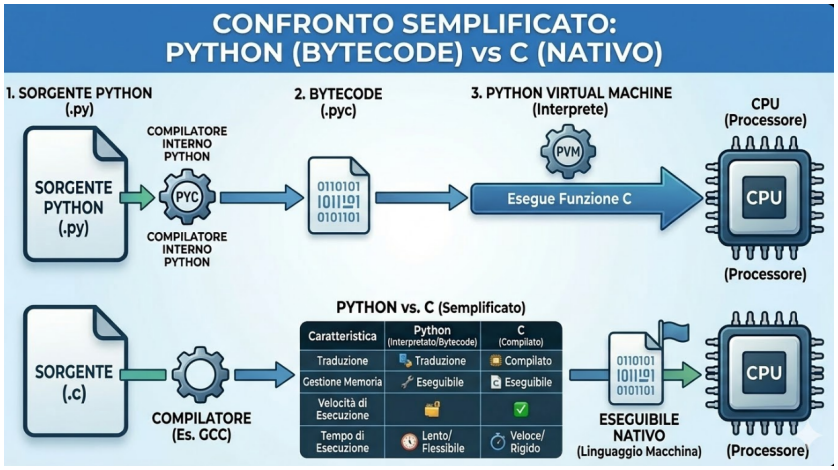


Figura 2: Confronto tra C e Python

Caratteristica	Linguaggio C	Python
Traduzione	Compilazione statica	Compilazione JIT/Bytecode
Eseguibile	Binario nativo (OS-specific)	Bytecode (Universale)
Astrazione	Bassa (vicino all'hardware)	Alta (astrazione totale)
Gestione Memoria	Manuale (`malloc/free`)	Automatica (Garbage Collector)

Tabella 1: Differenze architetturali principali.

2.2 Esempio

Supponiamo di avere il seguente codice python:

```
Week1 > utility.py > ...
1  def sum(a, b):
2      print('ciao')
3      return a + b
4
5
6  import dis      # Disassembler of Python byte code into mnemonics.
7
8  dis.dis(sum)    # dis.dis(...) Disassemble classes, methods, functions, and other compiled objects.
9                  # In this case, the sum function
10
```

Il modulo *dis* è il disassemblatore. Il suo compito è prendere il codice Python (che è scritto in un linguaggio comprensibile agli umani) e mostrarti le istruzioni a basso livello che la Python Virtual Machine (PVM) esegue effettivamente "sotto il cofano".

L'output è il seguente:

```
PS C:\Users\miche\Desktop\GitRepositories\DesignAnalysisOfAlgorithms\Week1> python utility.py
  2           0 LOAD_GLOBAL              0 (print)
              2 LOAD_CONST               1 ('ciao')
              4 CALL_FUNCTION            1
              6 POP_TOP

  3           8 LOAD_FAST                0 (a)
             10 LOAD_FAST                1 (b)
             12 BINARY_ADD
             14 RETURN_VALUE
```

Il blocco `print('ciao')` (Righe 0-6) Qui Python sta preparando una chiamata a funzione:

- **LOAD_GLOBAL 0 (print)**: Python deve andare a cercare la funzione print. Dato che non è definita dentro la tua funzione sum, la cerca nello spazio "Globale" (o nei built-ins).
- **LOAD_CONST 1 ('ciao')**: Carica l'argomento da passare alla funzione.
- **CALL_FUNCTION 1: Dice alla PVM**: "*Prendi la funzione che hai caricato, passale 1 argomento e avviala*". Qui la PVM sospende per un attimo sum e va a eseguire il codice C di print.

- **POP_TOP**: Questa è fondamentale. La funzione print restituisce sempre None, che viene messo sullo stack. Dato che non stai salvando il risultato di print in una variabile (es. non hai scritto x = print('ciao')), Python "pulisce" lo stack eliminando quel None.

Il blocco `return a + b` (Righe 8-14)

- **LOAD_FAST 0 (a) e 1 (b)**: Python carica i valori dei parametri locali. L'uso di *FAST* indica che l'accesso avviene tramite un indice in un array locale, rendendolo molto più rapido rispetto alla ricerca in un dizionario globale.

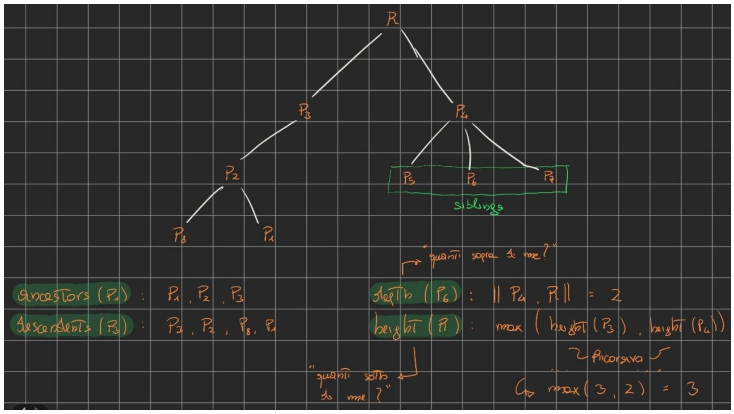
- **BINARY_ADD**: L'istruzione preleva i due valori dallo stack e invoca la routine C corrispondente per eseguire l'addizione. Il risultato viene poi riposto in cima allo stack.

- **RETURN_VALUE**: Conclude l'esecuzione della funzione restituendo al chiamante l'elemento attualmente presente in cima allo stack.

2.3 Conclusione

Sebbene l'interposizione della PVM renda Python intrinsecamente più lento del C, questo approccio permette una gestione della memoria semplificata, una portabilità cross-platform immediata e una velocità di sviluppo superiore, delegando le operazioni critiche a routine interne pre-ottimizzate in C.

3 Trees



- Ogni nodo, escluso il nodo in alto (Radice), ha un padre e un numero arbitrario di figli.
- Due nodi che sono figli dello stesso padre sono frtelli;
- Un nodo è una foglia se non ha figli.
- Un nodo è interno se ha almeno un figlio.
- un nodo v è un **antenato** di un nodo u se $u = v$ o u è un antenato del padre di v.
- Un nodo v è un **descendente** di un nodo u se u è un antenato di v.
- Un **edge** è una coppia di nodi *(u, v)* tale che u è il padre di v o viceversa.
- Un **path** è una sequenza di nodi tali che qualsiasi coppia di nodi consecutivi è un **edge**.

3.1 Abstract Data Type

structure. A position object for a tree supports the method:

p.element(): Return the element stored at position p.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree:

T.root(): Return the position of the root of tree T, or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p, or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that are contained in tree T.

T.is_empty(): Return True if tree T does not contain any positions.

T.positions(): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.